



Made by Star Banner Games.

Itch.io: [Star Banner Games](#)

LinkedIn: [Sanandana Michael Ithaler](#)

Twitter: [@starbannergames](#)

Content

1	QUICKSTART	1
1.1	Overview	1
1.2	QuickTimer	1
1.3	QueueTimer	2
2	ADVANCED GUIDE	3
2.1	Overview	3
2.1.1	How it works	3
2.1.2	Delegate Callbacks (Help for Beginners).....	3
2.2	QuickTimer	4
2.2.1	StartTimer	4
2.2.2	AbortTimer	4
2.3	QueueTimer	5
2.3.1	RunningTimer	5
2.3.2	QueueLength.....	5
2.3.3	EnqueueTimer.....	5
2.3.4	AbortRunningTimer	5
2.3.5	AbortAll.....	5
2.4	TimerData.....	6
2.4.1	TimerRoutine	6
2.4.2	StartTime.....	6
2.4.3	RemainingSeconds.....	6
2.4.4	LifetimeInSeconds	6
2.4.5	SourceBehaviour.....	6
2.4.6	DurationInSeconds.....	6
2.4.7	Callback.....	6
2.4.8	UsingUnscaledTime	6
2.4.9	IsRepeating	6

1 QuickStart

1.1 Overview

Pocketwatch contains two kinds of timers: **QuickTimers** and **QueueTimers**. While QuickTimers are meant for simple Delays, it is possible to string QueueTimers together to do a number of actions sequentially.

To use any of the functionality, open or create the **MonoBehaviour** Script on which you want to run a timer and include **SBG.Pocketwatch**.

1.2 QuickTimer

To start a QuickTimer, go to the desired position and call "**this.StartTimer()**", which takes a delegate for a callback. If you are new to delegates, please refer to the **Advanced Guide** below.

```
void Start()
{
    //Start a 5 seconds long timer
    this.StartTimer(() =>
    {
        //After the timer runs out, print "Timer finished!"
        Debug.Log("Timer finished!");
    }, 5f);
}
```

If you want to abort a running QuickTimer, you need to cache a timer at creation and call "**QuickTimer.AbortTimer()**".

```
void Start()
{
    //Start a timer and caches a reference to it.
    TimerData myTimer = this.StartTimer(() =>
    {
        Debug.Log("Timer finished!");
    }, 5f);

    //Abort the cached timer
    QuickTimer.AbortTimer(myTimer);
}
```

1.3 QueueTimer

To start a QueueTimer, first create a QueueTimer variable and then add timers to it using “**QueueTimer.EnqueueTimer()**”, which takes a delegate for a callback. If you are new to delegates, please refer to the **Advanced Guide** below.

```
void Start()
{
    //Creating a QueueTimer requires a Source MonoBehaviour to run its timers on.
    QueueTimer queueTimer = new QueueTimer(this);

    //The first timer will log after 1 second.
    queueTimer.EnqueueTimer(() =>
    {
        Debug.Log("Timer 1 finished!");
    }, 1f);

    //The second timer will log after 2 seconds.
    queueTimer.EnqueueTimer(() =>
    {
        Debug.Log("Timer 2 finished!");
    }, 1f);

    //The third timer will log after 3 seconds.
    queueTimer.EnqueueTimer(() =>
    {
        Debug.Log("Timer 3 finished!");
    }, 1f);
}
```

When using a QueueTimer, you can either abort the currently running timer, or the entire queue at once. Please see the example below:

```
void Start()
{
    QueueTimer queueTimer = new QueueTimer(this);

    //...Start a bunch of timers here...

    //Stop the timer currently running
    queueTimer.AbortRunningTimer();

    //Stop all timers in the Queue
    queueTimer.AbortAll();
}
```

If this Asset is useful to you, please consider rating it. Thank you!

2 Advanced Guide

2.1 Overview

2.1.1 How it works

Pocketwatch is based on three classes. **QuickTimer**, **QueueTimer** and **TimerData**. **QuickTimer** is used to create simple timers with callbacks. **QueueTimer** is based on creating an instance that has a queue of timers. This way you can chain timers together and make sure they are executed in the right order. Lastly, **TimerData** is meant to keep track of any of these timers to read out data such as the remaining time or the total duration. Under the hood, all timers are Coroutines, which means they require a **MonoBehaviour** to execute.

2.1.2 Delegate Callbacks (Help for Beginners)

Pocketwatch uses **delegates** for callbacks. If you are a beginner or just don't know how to use delegates, here's how it works in this context:

When your timer has finished, the callback will be invoked. If you just want to perform a simple action, you could write a function without any parameters and put "**YourFunctionName**" as the callback. If it needs to be more dynamic – or you want to save time – you can use a Lambda expression. The syntax can be confusing, but in this case, it is essentially just like a function that you write directly as a parameter. Simply look at the example below.

```
void Start()
{
    Action myDelegate = () =>
    {
        Debug.Log("Hello");
        Debug.Log("World");
    };
}
```

2.2 QuickTimer

2.2.1 StartTimer

This function will start a timer with the defined parameters. If you are calling this function on a `MonoBehaviour`, you can simply call `"this.StartTimer()"`. If you want to call this function elsewhere, or simply want to run it on a different `MonoBehaviour`, you will have to explicitly define that as the first parameter and use `"QuickTimer.StartTimer()"` instead.

Parameters:

- **SourceBehaviour (MonoBehaviour):** The `MonoBehaviour` that will start the timer's Coroutine.
- **Callback (Action):** Will be called at the end of the timer.
- **DurationInSeconds (float):** The length of the timer.
- **UnscaledTime (bool):** If true, the timer will be unaffected by timescale.
- **IsRepeating (bool):** If true, the timer will repeat until it is manually stopped. The Callback will be called at the end of each iteration.

Returns: A `TimerData` object referencing the started timer.

```
//Starts a 3 second long timer on this MonoBehaviour
this.StartTimer(SomeFunction, 3f, false, false);

//Starts a 3 second long timer on a different MonoBehaviour
QuickTimer.StartTimer(otherBehaviour, SomeFunction, 3f, false, false);
```

2.2.2 AbortTimer

This function will abort the given timer. No callbacks will be called. In order to use this, you need to get a reference to a `TimerData`, as shown in the example below.

Parameters:

- **Timer (TimerData):** The timer you want to abort.

```
void Start()
{
    //Start a timer and caches a reference to it.
    TimerData myTimer = this.StartTimer(() =>
    {
        Debug.Log("Timer finished!");
    }, 5f);

    //Abort the cached timer
    QuickTimer.AbortTimer(myTimer);
}
```

2.3 QueueTimer

In order to use a QueueTimer you need to instantiate one using the constructor.

Parameters:

- **SourceBehaviour:** The MonoBehaviour that will run the timers of this QueueTimer.

```
QueueTimer queueTimer = new QueueTimer(this);
```

2.3.1 RunningTimer

A reference to the **TimerData** that is currently running in the queue. May be **null**.

2.3.2 QueueLength

An **integer** describing the number of timers in this queue.

2.3.3 EnqueueTimer

This function will enqueue a timer with the defined parameters. Once it has reached the first place in the queue, the timer will execute.

Parameters:

- **Callback (Action):** Will be called at the end of the timer.
- **DurationInSeconds (float):** The length of the timer.
- **UnscaledTime (bool):** If true, the timer will be unaffected by timescale.
- **IsRepeating (bool):** If true, the timer will repeat until it is manually stopped. The Callback will be called at the end of each iteration.

```
///Start a QueueTimer that will complete its  
///Queue in 9 seconds total (7+2)  
  
QueueTimer queueTimer = new QueueTimer(this);  
queueTimer.EnqueueTimer(FirstFunction, 7f);  
queueTimer.EnqueueTimer(SecondFunction, 2f);
```

2.3.4 AbortRunningTimer

This Function aborts the currently running timer. If there are more timers in the queue, the next one will be triggered.

2.3.5 AbortAll

This Function will abort the entire queue, including the currently running timer.

2.4 TimerData

2.4.1 TimerRoutine

A reference to the **Coroutine** that is executing this timer.

2.4.2 StartTime

A **float** describing when this timer was started. If a timer is set to repeat, the start time will reset on each iteration.

2.4.3 RemainingSeconds

A **float** describing the remaining seconds until this timer ends. If a timer is set to repeat, this references only the current iteration.

2.4.4 LifetimeInSeconds

A **float** describing how long this timer has been running. If a timer is set to repeat, this references only the current iteration.

2.4.5 SourceBehaviour

The **MonoBehaviour** that runs the Timer's Coroutine.

2.4.6 DurationInSeconds

A **float** describing the duration of the timer.

2.4.7 Callback

An **Action** that will be called when this timer (or the current iteration) is finished.

2.4.8 UsingUnscaledTime

A **bool** describing whether this timer uses unscaled time.

2.4.9 IsRepeating

A **bool** describing whether this timer should loop after completion. A repeating timer will run until it is aborted.